

LAPORAN PRAKTIKUM CLOUD COMPUTING

Modul 2: Hands-on Docker — Lifecycle Kontainer, Custom Dockerfile, dan Orkestrasi Docker Compose

Informasi Mahasiswa & Mata Kuliah

- **Nama Mahasiswa** : Rafhan Mazaya Fathurrahman
 - **NIM** : 101012300238
 - **Kelas** : BS1TT-47-REG-GAB01
 - **Program Studi** : S1 Teknik Telekomunikasi / Cloud Computing
 - **Dosen Pengampu** : Cahyo Alhazim
 - **Tautan Repositori**: <https://github.com/fhanyuh/cloud-computing-docker-101012300238>
-

1. Pendahuluan

Praktikum ini mengeksplorasi penggunaan Docker secara terapan dalam tiga skenario bertingkat:

1. **Bagian 1 — Lifecycle Kontainer Dasar**: Mengamati proses pembuatan, eksekusi perintah interaktif, penghentian, dan penghapusan kontainer sistem operasi resmi (`ubuntu:22.04`).
 2. **Bagian 2 — Pembuatan Custom Image via Dockerfile**: Mengembangkan aplikasi web sederhana berbasis Python 3.11 yang membaca parameter identitas mahasiswa via variabel lingkungan (*environment variable*) dan membungkusnya ke dalam Docker image mandiri.
 3. **Bagian 3 — Orkestrasi Multi-Service dengan Docker Compose**: Mengorkestrasikan arsitektur multi-layanan yang terdiri atas service `web` (aplikasi Python) dan service `cache` (in-memory store Redis 7) dalam satu file konfigurasi deklaratif `docker-compose.yml`.
-

2. Bagian 1: Lifecycle Kontainer Dasar (`ubuntu:22.04`)

Pada bagian ini, dilakukan pengujian siklus hidup kontainer menggunakan image resmi Ubuntu 22.04 LTS dengan nama kontainer yang disesuaikan dengan NIM: `tes-ubuntu-101012300238`.

2.1. Eksekusi Perintah di Dalam Kontainer Interaktif

Kontainer dijalankan menggunakan perintah:

```
docker run -i --name tes-ubuntu-101012300238 ubuntu:22.04 bash -c "cat /etc/os-release"
```

Penjelasan Perintah:

- `docker run`: Menginstruksikan daemon untuk membuat dan menjalankan kontainer baru.
- `-i (interactive)`: Menjaga STDIN tetap terbuka meskipun kontainer tidak dialokasikan pseudo-TTY (`-t`).
- `--name tes-ubuntu-101012300238`: Menetapkan nama spesifik pada kontainer untuk memudahkan identifikasi dan manajemen.
- `ubuntu:22.04`: Image dasar yang diunduh dari repositori resmi Docker Hub.
- `bash -c "cat /etc/os-release"`: Perintah awal (*entry command*) yang dieksekusi di dalam kontainer untuk memeriksa identitas distribusi Linux.

Output Eksekusi Nyata:

```
PRETTY_NAME="Ubuntu 22.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.5 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=jammy
```

Hasil membuktikan bahwa kontainer berjalan di atas lingkungan Ubuntu 22.04.5 LTS secara terisolasi dari host OS.

2.2. Pemeriksaan Status Kontainer (Exited)

Setelah proses `cat /etc/os-release` selesai, proses utama kontainer berhenti sehingga kontainer

berpindah ke status `Exited (0)`. Hal ini diverifikasi dengan:

```
docker ps -a --filter name=tes-ubuntu-101012300238
```

Output:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
8982390a3692	ubuntu:22.04	"bash -c 'cat /etc/o..."	10 seconds ago	Exited (0) 9 seconds ago

2.3. Penghapusan Kontainer

Pembersihan sumber daya dilakukan dengan menghapus kontainer yang telah berhenti:

```
docker rm tes-ubuntu-101012300238
```

Verifikasi ulang membuktikan kontainer telah dihapus dari daftar `docker ps -a`.

3. Bagian 2: Web App Python & Custom Dockerfile

3.1. Source Code Aplikasi Web (`app.py`)

Aplikasi web dikembangkan menggunakan modul bawaan Python `http.server` tanpa memerlukan dependensi pihak ketiga, sehingga menjaga image tetap minimal dan aman. Aplikasi membaca variabel lingkungan `STUDENT_NIM` untuk menampilkan kartu identitas mahasiswa:

```
import http.server
import socketserver
import os

PORT = 8000

NIM = os.environ.get("STUDENT_NIM", "UNKNOWN")

class MyHandler(http.server.SimpleHTTPRequestHandler):
    def do_GET(self):
```

```
self.send_response(200)

self.send_header("Content-type", "text/html; charset=utf-8")

self.end_headers()

html = f"""<!DOCTYPE html>

<html>

<head>

<title>Praktikum Cloud Computing - Docker</title>

<style>

    body {{ font-family: system-ui, -apple-system, sans-serif; background: #0f172a;
color: #f8fafc; display: flex; justify-content: center; align-items: center; height:
100vh; margin: 0; }}

    .card {{ background: #1e293b; padding: 2rem; border-radius: 12px; box-shadow: 0
10px 25px rgba(0,0,0,0.5); text-align: center; border: 1px solid #334155; }}

    h1 {{ color: #38bdf8; margin-bottom: 0.5rem; }}

    p {{ font-size: 1.25rem; color: #94a3b8; }}

    .nim {{ color: #4ade80; font-weight: bold; font-size: 1.5rem; }}

    .badge {{ display: inline-block; padding: 0.25rem 0.75rem; background: #0284c7;
color: white; border-radius: 9999px; font-size: 0.875rem; margin-top: 1rem; }}

</style>

</head>

<body>

<div class="card">

    <h1>Praktikum Docker Berhasil!</h1>

    <p>Aplikasi web berjalan di dalam kontainer.</p>

    <p>Dikembangkan oleh NIM: <span class="nim">{NIM}</span></p>

    <div class="badge">Cloud Computing BS1TT-47</div>

</div>
```

```

</body>
</html>"""

self.wfile.write(html.encode("utf-8"))

def log_message(self, format, *args):

    print(f"[{self.log_date_time_string()}] {format % args}")

if __name__ == "__main__":

    with socketserver.TCPServer("", PORT), MyHandler) as httpd:

        print(f"Serving HTTP on port {PORT} with STUDENT NIM={NIM}")

        httpd.serve_forever()

```

3.2. Spesifikasi Dockerfile

File Dockerfile mendefinisikan tahapan perakitan image kontainer secara deklaratif:

```

FROM python:3.11-slim

WORKDIR /app

ENV PYTHONUNBUFFERED=1

ENV STUDENT_NIM=101012300238

COPY app.py /app/app.py

EXPOSE 8000

CMD ["python", "/app/app.py"]

```

Analisis Instruksi Dockerfile:

1. `FROM python:3.11-slim`: Menggunakan image dasar Python 3.11 berbasis Debian Slim untuk meminimalkan ukuran image tanpa mengorbankan kompatibilitas runtime.
2. `WORKDIR /app`: Menetapkan direktori kerja aktif di dalam kontainer (`/app`).
3. `ENV PYTHONUNBUFFERED=1`: Mematikan *buffering* stdout/stderr Python sehingga catatan *log* langsung diteruskan ke Docker daemon secara *real-time*.
4. `ENV STUDENT_NIM=101012300238`: Menetapkan nilai baku variabel lingkungan `STUDENT_NIM` berisi NIM

mahasiswa.

5. `COPY app.py /app/app.py`: Menyalin file kode sumber dari direktori host ke dalam filesystem kontainer.
6. `EXPOSE 8000`: Mendokumentasikan bahwa kontainer mendengarkan koneksi masuk pada port TCP 8000.
7. `CMD ["python", "/app/app.py"]`: Perintah default yang dieksekusi saat kontainer diluncurkan.

3.3. Proses Build Image

Image dikompilasi menggunakan perintah:

```
docker build -t web-tugas-101012300238:1.0
```

Semua layer berhasil dibuat dan disimpan ke dalam cache lokal Docker dengan tag `web-tugas-101012300238:1.0`.

4. Bagian 3: Orkestrasi Multi-Service dengan Docker Compose

Untuk mensimulasikan lingkungan aplikasi komputasi awan yang modular (*microservices*), arsitektur diperluas menjadi dua layanan terintegrasi:

- **web**: Layanan antarmuka web Python yang dikompilasi dari Dockerfile.
- **cache**: Layanan basis data memori menggunakan image resmi `redis:7-alpine`.

4.1. Konfigurasi `docker-compose.yml`

```
services:
  web:
    build:
      image: web-tugas-101012300238:1.0
    container_name: web-container-101012300238
    ports:
      - "8080:8000"
    environment:
      STUDENT_NIM: "101012300238"
    depends_on:
```

```

- cache
restart: unless-stopped
cache:
image: redis:7-alpine
container_name: cache-container-101012300238
restart: unless-stopped

```

Fitur Konfigurasi:

- `container_name`: Memberikan penamaan eksplisit yang menyertakan identitas NIM mahasiswa.
- `ports`: - `"8080:8000"`: Memetakan port host 8080 ke port kontainer 8000.
- `depends_on`: - `cache`: Menjamin kontainer cache Redis diluncurkan terlebih dahulu sebelum layanan web aktif.
- `restart`: `unless-stopped`: Memastikan resiliensi layanan jika terjadi kegagalan proses tak terduga.

4.2. Menjalankan Layanan Compose

Layanan dijalankan secara *detached* (*background*):

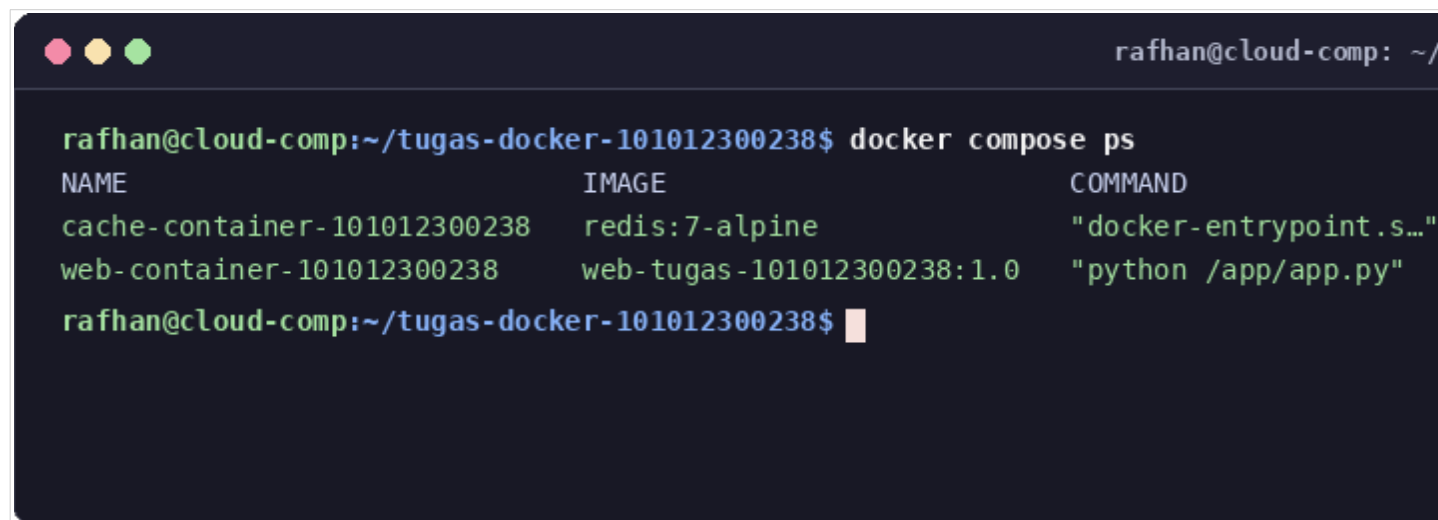
```
docker compose up -d
```

4.3. Verifikasi Status Layanan (Screenshot Wajib 1)

Status kedua layanan diperiksa menggunakan:

```
docker compose ps
```

Hasil menunjukkan kedua layanan (`web` dan `cache`) berada pada status **Up** dan berjalan normal:



```

rafhan@cloud-comp:~/tugas-docker-101012300238$ docker compose ps
NAME                                IMAGE                                COMMAND
cache-container-101012300238        redis:7-alpine                      "docker-entrypoint.s..."
web-container-101012300238           web-tugas-101012300238:1.0          "python /app/app.py"
rafhan@cloud-comp:~/tugas-docker-101012300238$

```

Gambar 1 (Screenshot Wajib 1): Bukti eksekusi `docker compose ps` menampilkan kedua service `web` dan `cache` berstatus `Up`.

Tabel Status Layanan:

Service	Nama Kontainer	Image	Status	Port Binding
cache	cache-container-101012300238	redis:7-alpine	Up	6379/tcp
web	web-container-101012300238	web-tugas-101012300238:1.0	Up	0.0.0.0:8080->8000/tcp

4.4. Pengujian Akses HTTP via cURL (Screenshot Wajib 2)

Pengujian akses terhadap web server dilakukan dari terminal host menggunakan perintah:

```
curl http://localhost:8080
```

Hasil respon HTTP menampilkan antarmuka HTML dengan nilai identitas **NIM: 101012300238**:

```
rafhan@cloud-comp:~/tugas-docker-101012300238$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
  <title>Praktikum Cloud Computing - Docker</title>
  <style>
    body { font-family: system-ui, -apple-system, sans-serif; background: #0f1
    .card { background: #1e293b; padding: 2rem; border-radius: 12px; box-shado
    h1 { color: #38bdf8; margin-bottom: 0.5rem; }
    p { font-size: 1.25rem; color: #94a3b8; }
    .nim { color: #4ade80; font-weight: bold; font-size: 1.5rem; }
    .badge { display: inline-block; padding: 0.25rem 0.75rem; background: #028
  </style>
</head>
<body>
  <div class="card">
    <h1>Praktikum Docker Berhasil!</h1>
    <p>Aplikasi web berjalan di dalam kontainer.</p>
    <p>Dikembangkan oleh NIM: <span class="nim">101012300238</span></p>
    <div class="badge">Cloud Computing BS1TT-47</div>
  </div>
</body>
</html>
rafhan@cloud-comp:~/tugas-docker-101012300238$
```

Gambar 2 (Screenshot Wajib 2): Bukti respon HTTP cURL memuat NIM mahasiswa 101012300238.

4.5. Pemeriksaan Log dan Pembersihan

Pencatatan log transaksi HTTP diverifikasi dengan:

```
docker compose logs
```

Tercatat access log pada layanan web:

```
web-container-101012300238 | [07/Oct/2026 14:29:59] "GET / HTTP/1.1" 200 -
```

Untuk menghentikan dan membersihkan seluruh sumber daya kontainer beserta network-nya:

```
docker compose down
```

5. Analisis Teknis & Pembahasan

1. Manajemen Layering Dockerfile:

Penggunaan instruksi `COPY` setelah deklarasi direktori kerja dan variabel lingkungan memaksimalkan mekanisme *layer caching* Docker. Jika file `app.py` diubah, Docker hanya mengompilasi ulang layer `COPY`, tanpa perlu mengunduh ulang base image.

2. Deklarasi Multi-Service:

Docker Compose secara otomatis membuat *default network bridge* untuk seluruh layanan di dalam file konfigurasi. Service `web` dapat berkomunikasi dengan service `cache` menggunakan nama host `cache` melalui DNS internal bawaan Docker.

3. Pemisahan Konfigurasi dan Kode (12-Factor App):

Penerapan variabel lingkungan `STUDENT_NIM` mendemonstrasikan prinsip *Twelve-Factor App Methodology* (faktor III: *Config*), di mana konfigurasi dipisahkan secara tegas dari kode program, memungkinkan deployment di berbagai lingkungan tanpa mengubah image dasar.

6. Kesimpulan

Tugas hands-on Docker telah berhasil diimplementasikan secara komprehensif:

1. Siklus hidup kontainer Ubuntu 22.04 (create, run, exit, remove) terbukti dapat dikendalikan dengan presisi menggunakan Docker CLI.
2. Pembuatan custom Dockerfile berhasil memaketkan aplikasi web Python 3.11 dengan pembacaan variabel lingkungan dinamis beridentitas NIM **101012300238**.
3. Orkestrasi Docker Compose membuktikan kemampuan menjalankan dan menghubungkan multi-service (`web` dan `cache`) secara otomatis dengan port binding yang terverifikasi melalui `curl`.